# Mobile and Autonomous Robots (UE23CS343BB7)

# Jackfruit Problem

# (Mini-Project Report)

**Project Title: Traffic Signal Detection Robot**


**Student Name(s) & SRN(s):**

**1. SHARVANI (PES1UG23AM165)**

**2. NAVYASHREE (PES1UG23AM180)**

**3. NIDHI SHETTY (PES1UG23AM185)**

**4. REVATHI (PES1UG23AM233)**

**Instructor: Dr. Adithya Balasubramanyam**

## 1. Introduction

Provide a brief background of the project. Explain:

- Problem Statement: Robots operating in real-world environments must navigate safely in traffic scenarios. This project addresses the challenge of detecting and responding to traffic signs in real time using a ROS2-based autonomous robot with a live webcam feed.
- Importance: Traffic sign recognition is critical in autonomous navigation. It enables robots and self-driving vehicles to obey road rules, improving safety in mixed human-robot environments. The ability to interpret visual traffic cues is a foundational skill for any autonomous mobile robot.
- Solution Overview: A hybrid YOLOv8 + OpenCV detection pipeline classifies traffic signs from a live webcam feed and publishes movement commands (GO, STOP, LEFT, RIGHT, SLOW, WAIT) to a simulated differential-drive robot in Gazebo Harmonic via ROS2 Jazzy, all orchestrated through a single launch file.

## 2. Objective

Clearly state the objectives of the project:

- Detect traffic signs from a live camera feed in real time without manual re-training on the host machine, and map detected signs to robot movement commands.
- Expected Outcomes: A fully integrated ROS2 system where a simulated differential-drive robot in Gazebo Harmonic responds accurately to traffic signs (GO, STOP, LEFT, RIGHT, SLOW, WAIT) detected via webcam, with a single launch command starting all components.
- Scope: The project covers real-time sign detection (YOLOv8 + OpenCV), robot velocity control (Twist messages via /cmd_vel), Gazebo simulation with a custom urban road world, and full ROS2 integration using sign_detector, robot_controller, and ros_gz_bridge nodes.

## 3. Methodology

### 3.1 Algorithms

Explain the algorithms used in your project:

- Algorithms Used: (1) YOLOv8n (nano) — a deep learning object detection model (Ultralytics) trained on 1,165 traffic sign images across 56 classes; acts as the primary detector. (2) OpenCV HSV + approxPolyDP — colour segmentation in HSV space combined with contour-based shape classification (vertex count); acts as a fallback when YOLO confidence is low.
- Flow of Execution: Webcam frame captured by sign_detector node → YOLOv8 inference (primary) → OpenCV HSV/shape detection (fallback, for non-overlapping

regions only) → Best command selected → Published to /traffic_sign topic → robot_controller node reads command → Publishes Twist velocity to /cmd_vel → ROS-GZ Bridge forwards to Gazebo DiffDrive plugin → Robot moves accordingly.

- Justification: YOLOv8 is state-of-the-art for real-time object detection with excellent accuracy on custom datasets. The OpenCV fallback adds robustness when YOLO confidence is low due to lighting or partial occlusion. Using YOLO detections as primary and OpenCV as non-overlapping fallback avoids false positives from natural scene colours (e.g., green trees triggering GO).
- Include flowcharts or pseudo-code (Optional)

### 3.2 Tools Used

List and describe the tools, platforms, and technologies:

- Software: ROS2 Jazzy (Ubuntu 24.04 on VirtualBox), Python 3, Ultralytics YOLOv8 (YOLOv8n/YOLOv8s), OpenCV 4.x, colcon build tool, Google Colab (T4 GPU for model training).
- Hardware: Physical USB/built-in webcam for live sign detection; host machine running VirtualBox with Ubuntu 24.04. No additional hardware sensors on the robot (webcam replaces an on-board camera).
- Simulation Environment: Gazebo Harmonic (gz sim 8.11.0) with a custom urban road world (traffic_world.sdf) featuring a 60m road, pavements, buildings, and a differential-drive robot (robot.sdf) with DiffDrive plugin. Robot spawned at x = -20 m heading east.
- Libraries/Frameworks: Ultralytics (YOLOv8), OpenCV 4.x (with GTK support), ros-gz-bridge (ROS2 ↔ Gazebo bridge), Roboflow dataset (traffic-sign-umak0 v8, 56 classes, 1,165 images), NumPy (pinned <2 for compatibility).

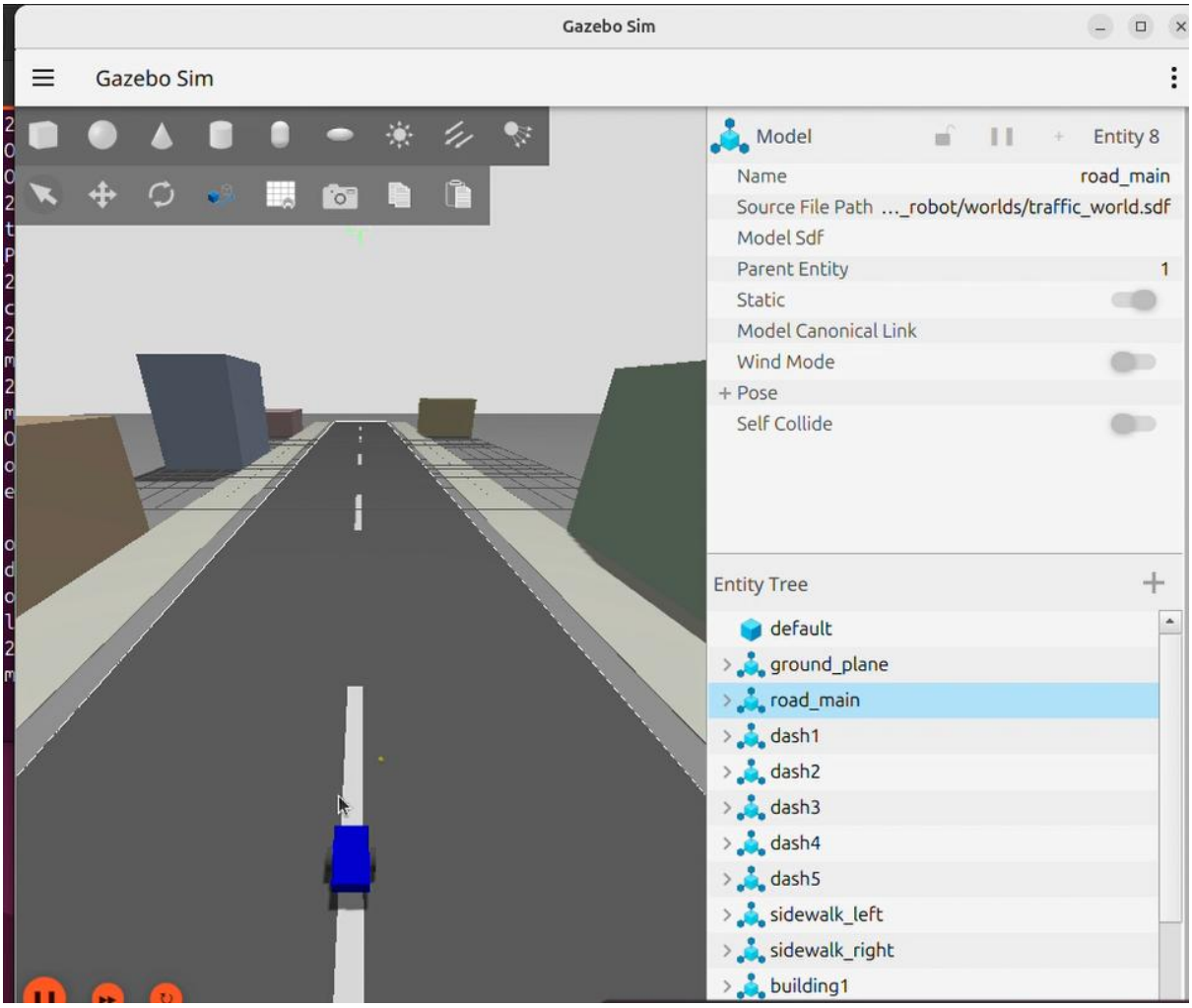
## 4. Outcome

The project successfully implements an autonomous traffic signal detection and response robot. The sign_detector node classifies signs via the hybrid YOLOv8 + OpenCV pipeline and publishes commands in real time. The robot_controller node maps commands to Twist velocities and drives the simulated robot accordingly. The full pipeline is operational from a single ROS2 launch command. The trained YOLOv8n model (30 epochs, 56 classes) is ready for deployment and the Gazebo world with road, buildings, and robot spawn point is fully built.

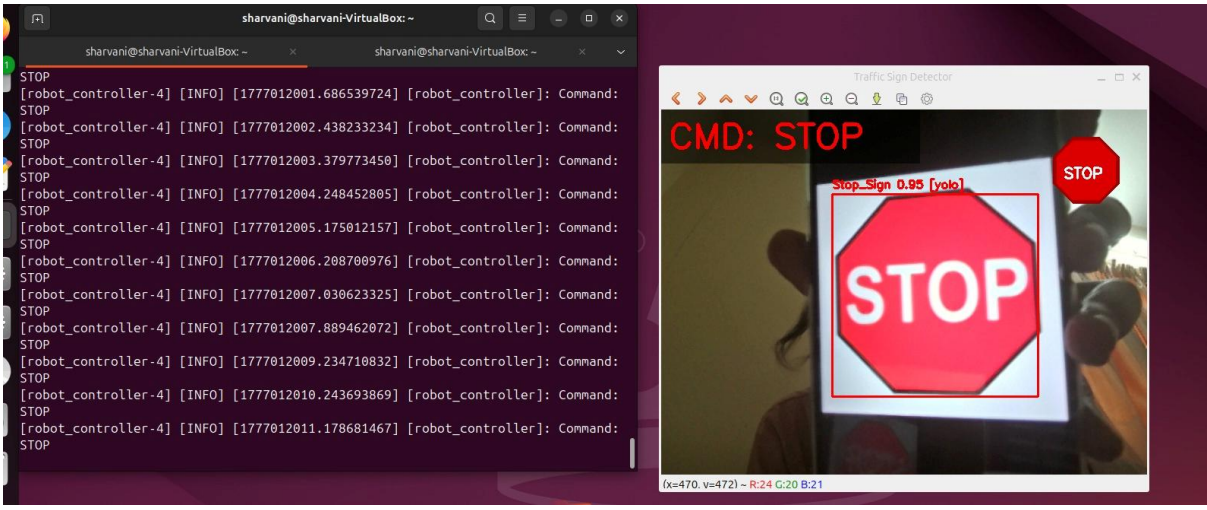
### 4.1 Result Screenshots

- Include clear screenshots of:
    - Simulation results

-
Traffic Sign Detec
CMD: STOP
Stop_Sign 0.93 [yolo]
STOP

- 


  - System outputs




- Add captions explaining each image

## 4.2 Video Demonstration

**Drive Link:** **MARS - Google Drive**

### 4.3 GitHub Repository

**GitHub Link:** Navyashree04/TRAFFIC-SIGNAL-DETECTION-ROBOT

## 5. Conclusion

Summarize:

- Key Learnings: Integrating deep learning (YOLOv8) with classical computer vision (OpenCV HSV + shape detection) produces a more robust pipeline than either approach alone. ROS2 launch files with staggered timers are essential for managing multi-node startup order. Training on a cloud GPU (Google Colab T4) reduced model training time from 4-12 hours on CPU to approximately 20 minutes. Proper file transfer methods (Google Drive vs. WhatsApp) are critical to avoid model corruption.
- Challenges Faced: (1) CPU training took 4-12 hours on the full 18,640-image dataset — resolved by switching to Google Colab T4 GPU with a reduced 1,165-image dataset. (2) Roboflow API returned 403 errors on public WiFi — resolved by using a mobile hotspot and training locally. (3) The best.pt model was corrupted when transferred via WhatsApp — resolved by using Google Drive for transfer. (4) NumPy 2.x was incompatible with pre-compiled OpenCV — resolved by pinning numpy<2. (5) Pure colour detection triggered on natural scene colours — resolved by combining YOLO (primary) with shape-constrained OpenCV (fallback).